

VL-13: Polynomielle Reduktionen

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

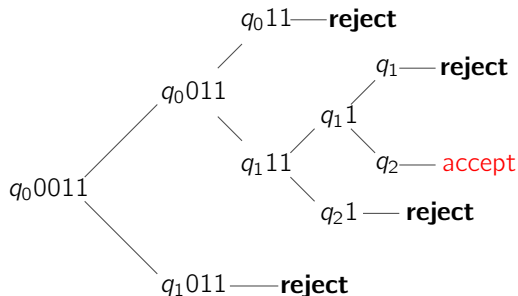
WS 2024, RWTH

Wiederholung

Wdh.: Non-deterministische Turingmaschine (NTM)

Definition: Akzeptanzverhalten der NTM

Eine NTM M akzeptiert die Eingabe $x \in \Sigma^*$,
falls es **mindestens** einen Rechenweg von M gibt,
der in eine Konfiguration mit akzeptierendem Zustand führt.



Komplexitätsklassen P und NP

P ist die Klasse aller Entscheidungsprobleme, die durch eine **TM** M erkannt werden, deren Worst Case Laufzeit $t_M(n)$ polynomiell beschränkt ist.

NP ist die Klasse aller Entscheidungsprobleme, die durch eine **NTM** M erkannt werden, deren Worst Case Laufzeit $t_M(n)$ polynomiell beschränkt ist.

Satz (Zertifikat Charakterisierung von NP)

Eine Sprache $L \subseteq \Sigma^*$ liegt genau dann in NP, wenn es einen polynomiellen **deterministischen** Algorithmus V und ein Polynom p mit der folgenden Eigenschaft gibt:

$$x \in L \iff \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$$

Definition (Postisches Korrespondenzproblem, PCP)

Eine Instanz des PCP besteht aus einer endlichen Menge

$$K = \left\{ \left[\frac{x_1}{y_1} \right], \dots, \left[\frac{x_k}{y_k} \right] \right\}.$$

Die Frage ist, ob es eine (nicht-leere) korrespondierende Folge $\langle i_1, \dots, i_n \rangle$ gibt, sodass $x_{i_1} x_{i_2} \dots x_{i_n} = y_{i_1} y_{i_2} \dots y_{i_n}$ gilt.

Ein zweifelhafter Satz

PCP $\in$ NP

Ein zweifelhafter Beweis:

- Zertifikat = korrespondierende Folge $\langle i_1, \dots, i_n \rangle$
- Verifizierer überprüft, ob $x_{i_1} x_{i_2} \dots x_{i_n} = y_{i_1} y_{i_2} \dots y_{i_n}$ gilt

Wdh.: Probleme in NP

Problem: Satisfiability (SAT)

Eingabe: Eine Boole'sche Formel φ in CNF über einer Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

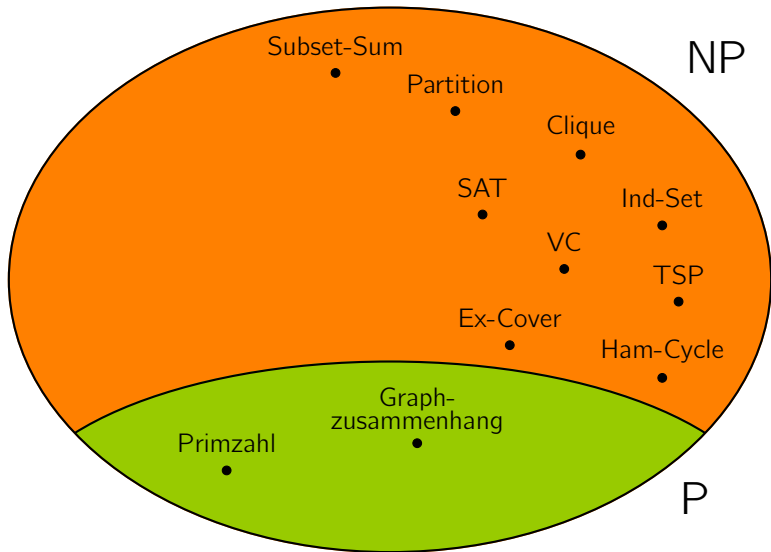
Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Problem: Hamiltonkreis (Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G einen Hamiltonkreis?

Wdh.: Die Komplexitätslandschaft



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.

P=NP ?

Falls die Lösung eines Problems einfach zu **überprüfen** ist,
ist es dann auch immer einfach, die Lösung selbst zu **entdecken**?

Vorlesung VL-13

Polynomielle Reduktionen

- Die Komplexitätsklasse EXPTIME
- Lösung finden versus Lösbarkeit entscheiden
- Optimieren versus Lösbarkeit entscheiden
- Polynomielle Reduktionen

Die Komplexitätsklasse EXPTIME

EXPTIME: Definition

Definition: Komplexitätsklasse EXPTIME

EXPTIME ist die Klasse aller Entscheidungsprobleme, die durch eine (deterministische) TM M entschieden werden, deren Worst Case Laufzeit durch $2^{q(n)}$ mit einem Polynom q beschränkt ist,

- Laufzeit-Beispiele: $2^{\sqrt{n}}$, 2^n , 3^n , $n \cdot 2^n$, $n!$, n^n
- Aber nicht: 2^{2^n} und n^{n^n}

EXPTIME: Definition

Definition: Komplexitätsklasse EXPTIME

EXPTIME ist die Klasse aller Entscheidungsprobleme, die durch eine (deterministische) TM M entschieden werden, deren Worst Case Laufzeit durch $2^{q(n)}$ mit einem Polynom q beschränkt ist,

- Laufzeit-Beispiele: $2^{\sqrt{n}}$, 2^n , 3^n , $n \cdot 2^n$, $n!$, n^n
- Aber nicht: 2^{2^n} und n^{n^n}

Frage: Wie verhalten sich die Klassen P und NP zu **EXPTIME**?

EXPTIME: $NP \subseteq EXPTIME$

Satz

$NP \subseteq EXPTIME$

Satz

$NP \subseteq EXPTIME$

- Es sei $L \in NP$
- Dann gibt es ein Polynom p und einen polynomiellen Algorithmus V
 $x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$

Satz

$NP \subseteq EXPTIME$

- Es sei $L \in NP$
- Dann gibt es ein Polynom p und einen polynomiellen Algorithmus V
 $x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$
- Wir enumerieren alle Kandidaten $y \in \{0,1\}^*$ mit $|y| \leq p(|x|)$.
Wir testen jeden einzelnen Kandidaten mit dem Verifizierer V .
Wir akzeptieren, falls V einen der Kandidaten akzeptiert.

Satz

$NP \subseteq EXPTIME$

- Es sei $L \in NP$
- Dann gibt es ein Polynom p und einen polynomiellen Algorithmus V
 $x \in L \iff \exists y \in \{0,1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x$
- Wir enumerieren alle Kandidaten $y \in \{0,1\}^*$ mit $|y| \leq p(|x|)$.
Wir testen jeden einzelnen Kandidaten mit dem Verifizierer V .
Wir akzeptieren, falls V einen der Kandidaten akzeptiert.
- Anzahl der Kandidaten $\approx 2^{p(|x|)}$
Zeit pro Kandidat $\approx$ polynomiell in $|x| + |y|$
- Ergo: Gesamtzeit $\approx \text{poly}(|x|) \cdot 2^{p(|x|)}$

EXPTIME: Zwei Beispiele

Problem: Satisfiability (SAT)

Eingabe: Eine Boole'sche Formel φ in CNF über der Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Problem: Hamiltonkreis (HAM-CYCLE)

Eingabe: Ein ungerichteter Graph $G = (V, E)$ mit $|V| = n$

Frage: Besitzt G einen Hamiltonkreis?

Frage: Welche (exponentielle) Zeitkomplexität ergibt sich aus dem vorangehenden Beweis für die Probleme SAT und HAM-CYCLE?

**Lösung finden
versus
Lösbarkeit entscheiden**

Lösung finden vs Lösbarkeit entscheiden

Ein beliebiges Entscheidungsproblem in NP

Eingabe: Ein diskretes Objekt X .

Frage: Existiert für dieses Objekt X ein Lösungsobjekt Y ?

Dilemma:

- Das Entscheidungsproblem beschäftigt sich nur mit der Frage, ob ein derartiges Lösungsobjekt Y **existiert**.
- Aber eigentlich will man das Lösungsobjekt Y auch genau **bestimmen**, und dann damit weiter arbeiten.

Ausweg:

- Ein schneller Algorithmus für das Entscheidungsproblem liefert (durch wiederholte Anwendung) oft auch einen schnellen Algorithmus zum Berechnen eines expliziten Lösungsobjekts

Beispiel: SAT (1)

Problem: Satisfiability (SAT)

Eingabe: Boole'sche Formel φ in CNF über $X = \{x_1, \dots, x_n\}$

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

- Wenn wir in φ eine Variable $x := 1$ setzen, so werden alle Klauseln mit Literal x dadurch erfüllt und in allen Klauseln mit Literal $\bar{x}$ fällt dieses Literal einfach weg.
- Wir erhalten eine kürzere CNF-Formel $\varphi[x = 1]$.
- Analog erhalten wir mit $x := 0$ die CNF-Formel $\varphi[x = 0]$.

Beispiel

Für $\varphi = (x \vee y \vee z) \wedge (\neg x \vee \neg y \vee \neg z) \wedge (\neg y \vee z) \wedge (u \vee z)$

gilt $\varphi[y = 1] = (\neg x \vee \neg z) \wedge (z) \wedge (u \vee z)$

und $\varphi[z = 0] = (x \vee y) \wedge (\neg y) \wedge (u)$

Beispiel: SAT (2)

Wir betrachten SAT Instanzen mit n Variablen und m Klauseln.

Satz

Angenommen, Algorithmus A entscheidet SAT Instanzen in $T(n, m)$ Zeit. Dann gibt es einen Algorithmus B , der für **erfüllbare** SAT Instanzen
in $n \cdot T(n, m)$ Zeit eine erfüllende Wahrheitsbelegung konstruiert.

Beispiel: SAT (2)

Wir betrachten SAT Instanzen mit n Variablen und m Klauseln.

Satz

Angenommen, Algorithmus A entscheidet SAT Instanzen in $T(n, m)$ Zeit. Dann gibt es einen Algorithmus B , der für **erfüllbare** SAT Instanzen in $n \cdot T(n, m)$ Zeit eine erfüllende Wahrheitsbelegung konstruiert.

Beweis:

- Wir fixieren der Reihe nach die Wahrheitswerte von $x_1, x_2, \dots, x_n$.
- FOR $i = 1, 2, \dots, n$ DO
 Wenn $\varphi[x_i = 1]$ erfüllbar, setze $x_i := 1$ und $\varphi := \varphi[x_i = 1]$
 Andernfalls setze $x_i := 0$ und $\varphi := \varphi[x_i = 0]$
- Am Ende ergeben die fixierten Wahrheitswerte von $x_1, x_2, \dots, x_n$ eine erfüllende Wahrheitsbelegung für φ

Beispiel: CLIQUE

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

Beispiel: CLIQUE

Problem: CLIQUE

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl k

Frage: Enthält G eine Clique mit $\geq k$ Knoten?

- Wenn wir aus G einen Knoten v und alle zu v inzidenten Kanten wegstreichen, so erhalten wir den kleineren Graphen $G - v$
- Falls $G - v$ eine k -Clique enthält, so ist v irrelevant
Falls $G - v$ keine k -Clique enthält, so muss v im Graphen bleiben

Satz

Angenommen, Algorithmus A entscheidet CLIQUE in $T(n)$ Zeit.
Dann gibt es einen Algorithmus B , der für JA-Instanzen
in $n \cdot T(n)$ Zeit eine k -Clique konstruiert.

Beispiel: Hamiltonkreis

Problem: Hamiltonkreis (Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G einen Hamiltonkreis?

Beispiel: Hamiltonkreis

Problem: Hamiltonkreis (Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G einen Hamiltonkreis?

- Wenn wir aus G eine Kante e wegstreichen, so erhalten wir den kleineren Graphen $G - e$
- Falls $G - e$ einen Hamiltonkreis enthält, so ist e irrelevant
Falls $G - e$ keinen Hamiltonkreis enthält, so muss e bleiben

Satz

Angenommen, Algorithmus A entscheidet Ham-Cycle in $T(n)$ Zeit.
Dann gibt es einen Algorithmus B , der für JA-Instanzen
in $|E| \cdot T(n)$ Zeit einen Hamiltonkreis konstruiert.

**Optimieren
versus
Lösbarkeit entscheiden**

Definition: Optimierungsproblem

Die Eingabe eines Optimierungsproblems spezifiziert (implizit oder explizit) eine Menge $\mathcal{L}$ von zulässigen Lösungen zusammen mit einer Zielfunktion $f : \mathcal{L} \rightarrow \mathbb{N}$, die Kosten, Gewicht, oder Profit misst.

Das Ziel ist es, eine optimale Lösung in $\mathcal{L}$ zu berechnen:

In **Minimierungs**problemen sollen die Kosten **minimiert** werden.

In **Maximierungs**problemen soll der Profit **maximiert** werden.

Dilemma:

Die Klassen P und NP enthalten keine Optimierungsprobleme, sondern nur Entscheidungsprobleme

Ausweg:

Jedes Optimierungsproblem kann in ein “**sehr ähnliches**” Entscheidungsproblem umformuliert werden

Beispiel: Travelling Salesman

- Beim Travelling Salesman Problem sind Städte $1, \dots, n$ gegeben, zusammen mit Distanzen $d(i, j)$ für $1 \leq i \neq j \leq n$
- Gesucht ist eine möglichst kurze Rundreise (Hamiltonkreis; Tour) durch alle Städte

Problem: Travelling Salesman (TSP)

Eingabe: Natürliche Zahlen $d(i, j)$ für $1 \leq i \neq j \leq n$

Zulässige Lösung: Permutation π von $1, \dots, n$

Ziel: Minimiere $d(\pi) := \sum_{i=1}^{n-1} d(\pi(i), \pi(i+1)) + d(\pi(n), \pi(1))$

Beispiel: Travelling Salesman

- Beim Travelling Salesman Problem sind Städte $1, \dots, n$ gegeben, zusammen mit Distanzen $d(i, j)$ für $1 \leq i \neq j \leq n$
- Gesucht ist eine möglichst kurze Rundreise (Hamiltonkreis; Tour) durch alle Städte

Problem: Travelling Salesman (TSP)

Eingabe: Natürliche Zahlen $d(i, j)$ für $1 \leq i \neq j \leq n$

Zulässige Lösung: Permutation π von $1, \dots, n$

Ziel: Minimiere $d(\pi) := \sum_{i=1}^{n-1} d(\pi(i), \pi(i+1)) + d(\pi(n), \pi(1))$

Entscheidungsproblem:

- Die Eingabe enthält zusätzlich eine Schranke γ
- Frage: Existiert eine zulässige Lösung mit Länge $d(\pi) \leq \gamma$?

Beispiel: Bin Packing

Beim Bin Packing sollen n Objekte $1, \dots, n$ mit Gewichten $w_1, \dots, w_n$ auf möglichst wenige Kisten mit Gewichtslimit B verteilt werden.

Problem: Bin Packing (BPP)

Eingabe: Eine ganze Zahl B und Zahlen $w_1, \dots, w_n \in \{1, \dots, B\}$

Zulässige Lösung: Partition von $1, \dots, n$ in Teilmengen $T_1, \dots, T_k$,
sodass $w(T_j) \leq B$ für alle $1 \leq j \leq k$ gilt.

Ziel: Minimiere die Anzahl k der Teilmengen.

Beispiel: Bin Packing

Beim Bin Packing sollen n Objekte $1, \dots, n$ mit Gewichten $w_1, \dots, w_n$ auf möglichst wenige Kisten mit Gewichtslimit B verteilt werden.

Problem: Bin Packing (BPP)

Eingabe: Eine ganze Zahl B und Zahlen $w_1, \dots, w_n \in \{1, \dots, B\}$

Zulässige Lösung: Partition von $1, \dots, n$ in Teilmengen $T_1, \dots, T_k$, sodass $w(T_j) \leq B$ für alle $1 \leq j \leq k$ gilt.

Ziel: Minimiere die Anzahl k der Teilmengen.

Entscheidungsproblem:

- Die Eingabe enthält zusätzlich eine Schranke γ für die Anzahl der Kisten.
- Frage: Existiert eine zulässige Lösung $T_1, \dots, T_k$ mit $k \leq \gamma$?

Beispiel: Knapsack

- Beim Knapsack Problem (KP; Rucksackproblem) sind n Objekte mit Gewichten $w_1, \dots, w_n$ und Profiten $p_1, \dots, p_n$ gegeben
- Ausserdem ist eine Gewichtsschranke $\bar{w}$ gegeben
- Wir suchen eine Teilmenge K der Objekte, die in einen Rucksack mit Gewichtsschranke $\bar{w}$ passt und die den Gesamtprofit maximiert

Problem: Knapsack (KP)

Eingabe: Natürliche Zahlen $w_1, \dots, w_n$, $p_1, \dots, p_n$, und b

Zulässige Lösung: Menge $K \subseteq \{1, \dots, n\}$ mit $w(K) := \sum_{i \in K} w_i \leq \bar{w}$

Ziel: Maximiere $p(K) := \sum_{i \in K} p_i$

Entscheidungsproblem:

- Die Eingabe enthält zusätzlich eine Schranke γ für den Profit
- Frage: Existiert eine zulässige Lösung K mit $p(K) \geq \gamma$?

Allgemein: Optimieren vs Entscheiden

Für ein Optimierungsproblem mit einer Menge $\mathcal{L}$ von zulässigen Lösungen und einer Kostenfunktion $f : \mathcal{L} \rightarrow \mathbb{N}$ definieren wir das entsprechende Entscheidungsproblem:

Eingabe: Wie im Optimierungsproblem; plus Schranke $\gamma \in \mathbb{N}$

Frage: Existiert eine zulässige Lösung $x \in \mathcal{L}$

mit $f(x) \geq \gamma$ (für Maximierungsprobleme) respektive

mit $f(x) \leq \gamma$ (für Minimierungsprobleme)?

Allgemein: Optimieren vs Entscheiden

Für ein Optimierungsproblem mit einer Menge $\mathcal{L}$ von zulässigen Lösungen und einer Kostenfunktion $f : \mathcal{L} \rightarrow \mathbb{N}$ definieren wir das entsprechende Entscheidungsproblem:

Eingabe: Wie im Optimierungsproblem; plus Schranke $\gamma \in \mathbb{N}$

Frage: Existiert eine zulässige Lösung $x \in \mathcal{L}$
mit $f(x) \geq \gamma$ (für Maximierungsprobleme) respektive
mit $f(x) \leq \gamma$ (für Minimierungsprobleme)?

- Mit Hilfe eines Algorithmus, der das Optimierungsproblem löst, kann man immer das entsprechende Entscheidungsproblem lösen. **(Wie?)**
- Mit Hilfe eines Algorithmus, der das Entscheidungsproblem löst, kann man den optimalen Zielfunktionswert bestimmen (und oft auch die dazugehörige optimale Lösung finden).

Beispiel: Knapsack (1)

Eingabe: Natürliche Zahlen $w_1, \dots, w_n, p_1, \dots, p_n; \bar{w}; \gamma$

Zulässig: Menge $K \subseteq \{1, \dots, n\}$ mit $w(K) \leq \bar{w}$

Optimierung: Berechne zulässiges K mit maximalem $p(K)$

Entscheidung: Existiert zulässiges K mit $p(K) \geq \gamma$?

Beispiel: Knapsack (1)

Eingabe: Natürliche Zahlen $w_1, \dots, w_n, p_1, \dots, p_n; \bar{w}; \gamma$

Zulässig: Menge $K \subseteq \{1, \dots, n\}$ mit $w(K) \leq \bar{w}$

Optimierung: Berechne zulässiges K mit maximalem $p(K)$

Entscheidung: Existiert zulässiges K mit $p(K) \geq \gamma$?

Satz

Wenn Knapsack-Entscheidung in $T(|I|)$ Zeit lösbar ist,
so ist Knapsack-Optimierung in $\text{poly}(|I|) \cdot T(|I|)$ Zeit lösbar.

Beweis: Aus einem schnellen Algorithmus A fürs Entscheidungsproblem

- konstruieren wir zuerst einen schnellen Algorithmus B ,
der den **optimalen Zielfunktionswert** bestimmt
- und dann einen schnellen Algorithmus C ,
der eine **optimale zulässige Lösung** bestimmt

Beispiel: Knapsack (2)

Algorithmus B für (Phase 1)

Wir führen eine **Binäre Suche** mit den folgenden Parametern durch:

- Der minimal mögliche Profit ist 0 .
- Der maximal mögliche Profit ist $P := \sum_{i=1}^n p_i$.
- Wir finden den optimalen Zielfunktionswert durch Binäre Suche über dem Wertebereich $\{0, \dots, P\}$.
- In jeder Iteration verwenden wir den polynomiellen Algorithmus A (für das Entscheidungsproblem), der uns mitteilt, in welcher Hälfte des verbleibenden Wertebereichs wir weitersuchen müssen

Die Anzahl der Iterationen der Binärsuche ist $\lceil \log(P + 1) \rceil$.

Beispiel: Knapsack (3)

- Algorithmus B besteht aus $\leq \lceil \log(P + 1) \rceil$ Aufrufen von Algorithmus A in der binären Suche.
- Also ist die Gesamtlaufzeit von Algorithmus B durch $\lceil \log(P + 1) \rceil \cdot T(|I|)$ beschränkt.

Beispiel: Knapsack (3)

- Algorithmus B besteht aus $\leq \lceil \log(P + 1) \rceil$ Aufrufen von Algorithmus A in der binären Suche.
- Also ist die Gesamtlaufzeit von Algorithmus B durch $\lceil \log(P + 1) \rceil \cdot T(|I|)$ beschränkt.

Die Eingabelänge $|I|$ der Knapsack-Instanz I beträgt

$$\begin{aligned} |I| &\geq \sum_{i=1}^n \log(p_i + 1) = \log\left(\prod_{i=1}^n (p_i + 1)\right) \\ &\geq \log\left(\sum_{i=1}^n (p_i + 1)\right) \approx \log(P + 1) \end{aligned}$$

Die Gesamtlaufzeit von Algorithmus B ist durch $|I| \cdot T(|I|)$ beschränkt.

Beispiel: Knapsack (4)

Aus Algorithmus B konstruieren wir nun noch den Algorithmus C , der die optimale zulässige Lösung bestimmt:

Algorithmus C

```
1  K := {1, ..., n};
2  opt := Algo_B(K);
3  for i := 1 to n do
4      if Algo_B(K - {i}) == opt then K := K - {i};
5  return K
```

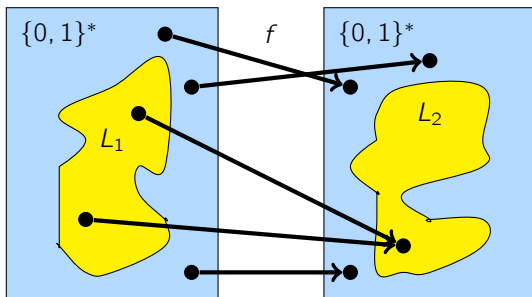
- Algorithmus C besteht im wesentlichen aus $n + 1$ Aufrufen des polynomiellen Algorithmus B
- Also ist die Gesamtlaufzeit von Algorithmus C durch $n \cdot |I| \cdot T(|I|)$ und folglich durch $\text{poly}(|I|) \cdot T(|I|)$ beschränkt

Polynomielle Reduktionen

Zur Erinnerung: Eine alte Seite aus Vorlesung VL-06

Definition

Es seien L_1 und L_2 zwei Sprachen über einem Alphabet Σ .
Dann ist L_1 auf L_2 reduzierbar (mit der Notation $L_1 \leq L_2$),
wenn eine berechenbare Funktion $f: \Sigma^* \rightarrow \Sigma^*$ existiert,
so dass für alle $x \in \Sigma^*$ gilt: $x \in L_1 \Leftrightarrow f(x) \in L_2$.



Polynomielle Reduktionen (1)

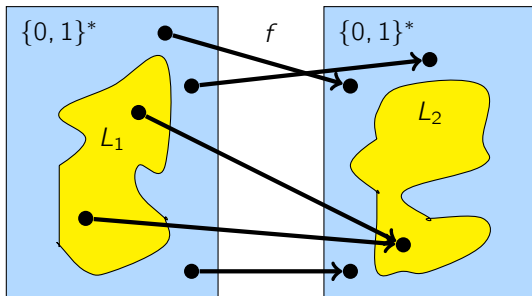
Definition

Es seien L_1 und L_2 zwei Sprachen über einem Alphabet Σ .

Dann ist L_1 **polynomiell** reduzierbar auf L_2 (mit der Notation

$L_1 \leq_p L_2$),

wenn eine **polynomiell** berechenbare Funktion $f: \Sigma^* \rightarrow \Sigma^*$ existiert, so dass für alle $x \in \Sigma^*$ gilt: $x \in L_1 \Leftrightarrow f(x) \in L_2$.



Polynomielle Reduktionen (2)

Satz

Falls $L_1 \leq_p L_2$ und falls $L_2 \in P$, so gilt $L_1 \in P$.

Polynomielle Reduktionen (2)

Satz

Falls $L_1 \leq_p L_2$ und falls $L_2 \in P$, so gilt $L_1 \in P$.

Beweis:

- Die Reduktion f hat eine polynomielle Laufzeitschranke $p(\cdot)$
- Ein Algorithmus A_2 entscheidet L_2 mit einer polynomiellen Laufzeitschranke $q(\cdot)$

Wir konstruieren einen Algorithmus A_1 , der L_1 entscheidet:

Schritt 1: Berechne $f(x)$

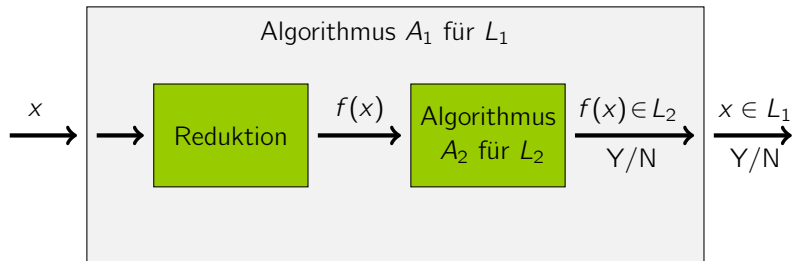
Schritt 2: Simuliere Algorithmus A_2 auf $f(x)$

Schritt 3: Akzeptiere x , genau dann wenn A_2 akzeptiert

Schritt 1 hat Laufzeit $p(|x|)$ und

Schritt 2 hat Laufzeit $q(|f(x)|) \leq q(p(|x|) + |x|)$

Polynomielle Reduktionen (3)



Beispiel zu Reduktionen:
COLORING $\leq_p$ SAT

COLORING $\leq_p$ SAT

Problem: Knotenfärbung / COLORING

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl $k \in \mathbb{N}$

Frage: Gibt es eine Färbung $c : V \rightarrow \{1, \dots, k\}$ der Knoten mit k Farben, sodass benachbarte Knoten verschiedene Farben erhalten? $\forall e = \{u, v\} \in E : c(u) \neq c(v)$?

Problem: Satisfiability (SAT)

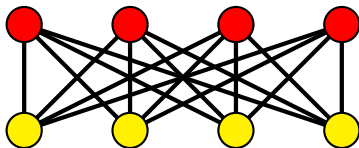
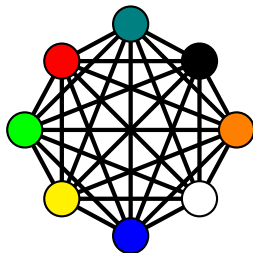
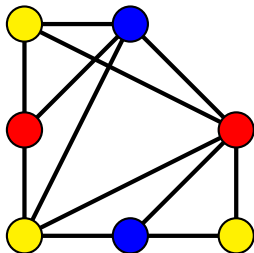
Eingabe: Boole'sche Formel φ in CNF über der Variablenmenge X

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Satz

COLORING $\leq_p$ SAT

Beispiel: Knotenfärbung von Graphen



COLORING $\leq_p$ SAT: Die Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Farbe $i \in \{1, \dots, k\}$ führen wir eine Boole'sche Variable x_v^i ein.

COLORING $\leq_p$ SAT: Die Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Farbe $i \in \{1, \dots, k\}$ führen wir eine Boole'sche Variable x_v^i ein.

Die Klauseln

Für jeden Knoten $v \in V$ verwenden wir die Klausel $(x_v^1 + x_v^2 + \dots + x_v^k)$

Für jede Kante $\{u, v\} \in E$ und jede Farbe $i \in \{1, \dots, k\}$ verwenden wir die Klausel $(\bar{x}_u^i + \bar{x}_v^i)$

COLORING $\leq_p$ SAT: Die Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Farbe $i \in \{1, \dots, k\}$ führen wir eine Boole'sche Variable x_v^i ein.

Die Klauseln

Für jeden Knoten $v \in V$ verwenden wir die Klausel $(x_v^1 + x_v^2 + \dots + x_v^k)$

Für jede Kante $\{u, v\} \in E$ und jede Farbe $i \in \{1, \dots, k\}$ verwenden wir die Klausel $(\bar{x}_u^i + \bar{x}_v^i)$

- Anzahl der Variablen = $k |V|$
- Anzahl der Klauseln = $|V| + k |E|$
- Gesamtlänge der Formel = $k |V| + 2k |E| \in O(k |V|^2)$

COLORING $\leq_p$ SAT: Korrektheit (1)

Graph G hat k -Färbung $\Rightarrow$ Formel φ ist erfüllbar

COLORING $\leq_p$ SAT: Korrektheit (1)

Graph G hat k -Färbung $\Rightarrow$ Formel φ ist erfüllbar

- Es sei c eine k -Färbung für G
- Für jeden Knoten v mit $c(v) = i$ setzen wir $x_v^i = 1$.
Alle anderen Variablen setzen wir auf 0.

COLORING $\leq_p$ SAT: Korrektheit (1)

Graph G hat k -Färbung $\Rightarrow$ Formel φ ist erfüllbar

- Es sei c eine k -Färbung für G
- Für jeden Knoten v mit $c(v) = i$ setzen wir $x_v^i = 1$.
Alle anderen Variablen setzen wir auf 0.
- Für jeden Knoten $v \in V$ ist $(x_v^1 + x_v^2 + \dots + x_v^k)$ erfüllt
- Für $\{u, v\} \in E$ und $i \in \{1, \dots, k\}$ ist $(\bar{x}_u^i + \bar{x}_v^i)$ erfüllt
Andernfalls hätten beide Knoten u und v die selbe Farbe i .

COLORING $\leq_p$ SAT: Korrektheit (1)

Graph G hat k -Färbung $\Rightarrow$ Formel φ ist erfüllbar

- Es sei c eine k -Färbung für G
- Für jeden Knoten v mit $c(v) = i$ setzen wir $x_v^i = 1$.
Alle anderen Variablen setzen wir auf 0.
- Für jeden Knoten $v \in V$ ist $(x_v^1 + x_v^2 + \dots + x_v^k)$ erfüllt
- Für $\{u, v\} \in E$ und $i \in \{1, \dots, k\}$ ist $(\bar{x}_u^i + \bar{x}_v^i)$ erfüllt
Andernfalls hätten beide Knoten u und v die selbe Farbe i .
- Ergo: Diese Wahrheitsbelegung erfüllt die Formel φ

COLORING $\leq_p$ SAT: Korrektheit (2)

Formel φ ist erfüllbar $\Rightarrow$ Graph G hat k -Färbung

COLORING $\leq_p$ SAT: Korrektheit (2)

Formel φ ist erfüllbar $\Rightarrow$ Graph G hat k -Färbung

- Wir betrachten eine beliebige erfüllende Belegung für φ
- Wegen der Klausel $(x_v^1 + x_v^2 + \dots + x_v^k)$ gibt es für jeden Knoten v mindestens eine Farbe i mit $x_v^i = 1$
- Für jeden Knoten wählen wir eine beliebige derartige Farbe aus

COLORING $\leq_p$ SAT: Korrektheit (2)

Formel φ ist erfüllbar $\Rightarrow$ Graph G hat k -Färbung

- Wir betrachten eine beliebige erfüllende Belegung für φ
- Wegen der Klausel $(x_v^1 + x_v^2 + \dots + x_v^k)$ gibt es für jeden Knoten v mindestens eine Farbe i mit $x_v^i = 1$
- Für jeden Knoten wählen wir eine beliebige derartige Farbe aus
- Wir behaupten: $c(u) \neq c(v)$ gilt für jede Kante $\{u, v\} \in E$
- Beweis: Falls $c(u) = c(v) = i$, dann gilt $x_u^i = x_v^i = 1$. Dann wäre aber die Klausel $(\bar{x}_u^i + \bar{x}_v^i)$ verletzt

COLORING $\leq_p$ SAT: Konsequenzen

Aus unserer Reduktion COLORING $\leq_p$ SAT folgt:

Folgerung 1

Wenn SAT einen polynomiellen Algorithmus hat,
so hat auch COLORING einen polynomiellen Algorithmus.

Folgerung 2 (logisch äquivalent zu Folgerung 1)

Wenn COLORING keinen polynomiellen Algorithmus hat,
so hat auch SAT keinen polynomiellen Algorithmus.

Übung: $\text{Ex-Cover} \leq_p \text{SAT}$

Problem: Exact Cover (Ex-Cover)

Eingabe: Eine endliche Menge X ; Teilmengen $S_1, \dots, S_m$ von X

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, m\}$,
sodass die Mengen S_i mit $i \in I$ eine Partition von X bilden?

Übung

Zeigen Sie: $\text{Ex-Cover} \leq_p \text{SAT}$

Beispiel zu Reduktionen:
Vertex Cover $\leq_p$ SAT

Vertex Cover $\leq_p$ SAT

Problem: Vertex Cover (VC)

Eingabe: Ein ungerichteter Graph $G = (V, E)$; eine Zahl $k \in \mathbb{N}$

Frage: Enthält G ein Vertex Cover mit $\leq k$ Knoten?

Vertex Cover $S \subseteq V$ enthält mindestens einen Endpunkt von jeder Kante

Problem: Satisfiability (SAT)

Eingabe: Boole'sche Formel φ in CNF über der Variablenmenge X

Frage: Existiert eine Wahrheitsbelegung von X , die φ erfüllt?

Satz

Vertex Cover $\leq_p$ SAT

Vertex Cover $\leq_p$ SAT: Eine zweifelhafte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$

führen wir eine Boole'sche Variable x_v ein.

(Variable x_v ist TRUE) $\iff$ (Knoten v ist im Vertex Cover)

Vertex Cover $\leq_p$ SAT: Eine zweifelhafte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$

führen wir eine Boole'sche Variable x_v ein.

(Variable x_v ist TRUE) $\iff$ (Knoten v ist im Vertex Cover)

Die Klauseln

Für jede Kante $\{u, v\} \in E$

verwenden wir die Klausel $(x_u + x_v)$

Für jede $(k + 1)$ -elementige Teilmenge $S \subseteq V$

verwenden wir die Klausel $\bigvee_{v \in S} \bar{x}_v$

Vertex Cover $\leq_p$ SAT: Eine zweifelhafte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$

führen wir eine Boole'sche Variable x_v ein.

(Variable x_v ist TRUE) $\iff$ (Knoten v ist im Vertex Cover)

Die Klauseln

Für jede Kante $\{u, v\} \in E$

verwenden wir die Klausel $(x_u + x_v)$

Für jede $(k+1)$ -elementige Teilmenge $S \subseteq V$

verwenden wir die Klausel $\bigvee_{v \in S} \bar{x}_v$

- Anzahl der Variablen = $|V|$
- Anzahl der Klauseln $\approx |V|^{k+1}$ (???????????)

Vertex Cover $\leq_p$ SAT: Eine korrekte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Zahl $i \in \{1, 2, \dots, k\}$
führen wir eine Boole'sche Variable x_v^i ein.

(Variable x_v^i ist TRUE) $\iff$ (v ist der i -te Knoten im Vertex Cover)

Vertex Cover $\leq_p$ SAT: Eine korrekte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Zahl $i \in \{1, 2, \dots, k\}$ führen wir eine Boole'sche Variable x_v^i ein.

(Variable x_v^i ist TRUE) $\iff$ (v ist der i -te Knoten im Vertex Cover)

Die Klauseln

Für jede Kante $\{u, v\} \in E$ verwenden wir die Klausel $(\bigvee_{1 \leq i \leq k} x_u^i \vee \bigvee_{1 \leq i \leq k} x_v^i)$

Für jede Zahl $i \in \{1, 2, \dots, k\}$ und je zwei Knoten $u, v \in V$ mit $u \neq v$ verwenden wir die Klausel $(\bar{x}_v^i + \bar{x}_u^i)$

Vertex Cover $\leq_p$ SAT: Eine korrekte Reduktion

Die Boole'schen Variablen

Für jeden Knoten $v \in V$ und für jede Zahl $i \in \{1, 2, \dots, k\}$ führen wir eine Boole'sche Variable x_v^i ein.

(Variable x_v^i ist TRUE) $\iff$ (v ist der i -te Knoten im Vertex Cover)

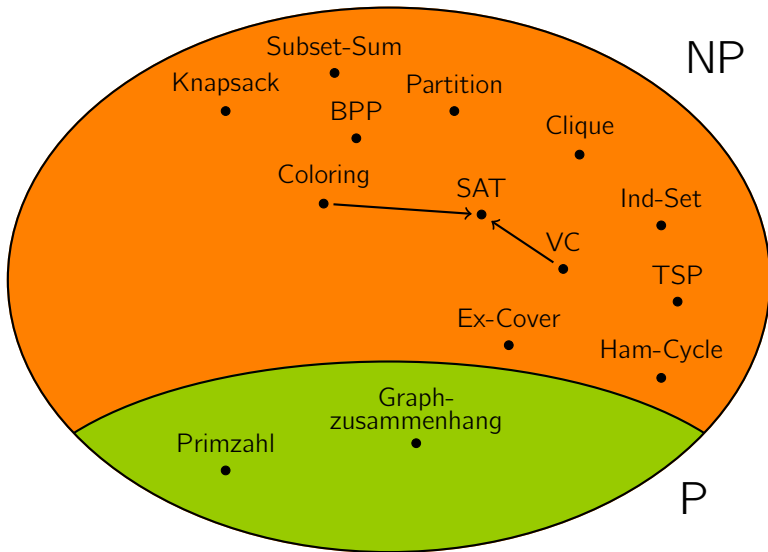
Die Klauseln

Für jede Kante $\{u, v\} \in E$ verwenden wir die Klausel $(\bigvee_{1 \leq i \leq k} x_u^i \vee \bigvee_{1 \leq i \leq k} x_v^i)$

Für jede Zahl $i \in \{1, 2, \dots, k\}$ und je zwei Knoten $u, v \in V$ mit $u \neq v$ verwenden wir die Klausel $(\bar{x}_v^i + \bar{x}_u^i)$

- Anzahl der Variablen = $k|V|$
- Anzahl der Klauseln $\approx |E| + k|V|^2$

Die Komplexitätslandschaft



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.